

SOFTWARE VERIFICATION AND VALIDATION

Design Document

SOFTWARE DESIGN SPECIFICATION (SDS)

Template • Based on IEEE 1016-2009 • For use after Requirements Verification

Project Title	[Enter project title here]
System Name	[Enter system or application name]
Document Version	v1.0
Date Prepared	[DD Month YYYY]
Prepared By	[Student Name(s) / Team Name]
Student Number(s)	[Student ID(s)]
Course	[Course Code] — Software Verification and Validation
Instructor	[Instructor Name]
Based On	Verified Requirements Specification (SRS) v__.

 This Software Design Specification (SDS) documents the architectural and detailed design decisions made to satisfy the verified requirements from the Software Requirements Specification (SRS). Complete every section. Replace all placeholder text shown in [brackets] or italicized gray with your actual content. Do not delete any section — write N/A with a brief justification if a section genuinely does not apply to your project.

DOCUMENT CONTROL

Revision History

 Record every version of this document. Begin with v1.0 at initial submission. Increment the minor version (v1.1, v1.2) for corrections and updates; increment the major version (v2.0) for significant structural revisions.

Version	Date	Author(s)	Description of Changes
v1.0	[Date]	[Author]	Initial release

Requirements Traceability Reference

 Identify the SRS version and date from which this design is derived. This links the design decisions in this document back to the verified requirements.

SRS Document	[Title and version of the verified Software Requirements Specification]
SRS Version	[e.g., SRS v2.1 — Final Verified]
SRS Date	[Date the SRS was verified and baselined]
Verification Method	[How requirements were verified: review, inspection, walkthrough, formal method]

1. Introduction

1.1 Purpose

 State the purpose of this document. Explain that it describes the design that will be implemented to satisfy the verified requirements, who the intended audience is, and how it should be used.

Purpose

This Software Design Specification (SDS) describes the architectural and detailed design of [System Name]. It serves as the technical blueprint that developers will follow during implementation. The primary audience is the development team, project supervisor, and quality assurance team. This document is derived from and traceable to the verified Software Requirements Specification (SRS) version [x.x].

1.2 Scope

 Define what software system this design covers. Identify what is in scope and, if relevant, what is explicitly out of scope.

Scope

[Describe the system being designed. What does it do at a high level? What are its boundaries? Which components or subsystems are included in this design? Are any components deferred to a future version?]

1.3 Definitions, Acronyms, and Abbreviations

 Define all technical terms, acronyms, and abbreviations used in this document. A reader unfamiliar with the project should be able to use this glossary.

Term / Acronym	Definition
SDS	Software Design Specification
SRS	Software Requirements Specification

API	Application Programming Interface
UI	User Interface
DB	Database
[Add more]	[Define this term]

1.4 References

 List all documents, standards, and sources referenced in this SDS. Include the verified SRS, any applicable standards (IEEE 1016, SWEBOK), and any third-party API or framework documentation used in the design.

References

1. [Project Name] Software Requirements Specification, Version [x.x], [Date] 2. IEEE 1016-2009: IEEE Standard for Information Technology — Systems Design — Software Design Descriptions 3. IEEE Computer Society. SWEBOK v3.0: Guide to the Software Engineering Body of Knowledge (2014) 4. [Framework/API name and version, URL or reference] 5. [Add additional references as needed]

1.5 Design Overview

 Provide a brief summary of the overall design approach — what architectural style was chosen (layered, MVC, microservices, event-driven, etc.), what the major components are, and how they relate to each other. This is a high-level overview; details appear in later sections.

Design Overview

[Summarize the design in 3–5 sentences. State the chosen architectural pattern, the major layers or components, and the key technology choices made. Example: 'The system follows a three-layer MVC architecture implemented in Python/Django. The presentation layer consists of a browser-based web interface. The business logic layer handles all application rules. Data is persisted in a PostgreSQL relational database.']

2. System Architecture

i Section 2 describes the high-level structure of the system — how it is divided into major components, how those components communicate, and why these choices were made. Every architectural decision should be traceable to one or more verified requirements.

2.1 Architectural Pattern

i Identify the architectural pattern(s) applied and justify the choice with reference to the verified requirements. Common patterns: Layered/N-tier, MVC (Model-View-Controller), Client-Server, Microservices, Event-Driven, Pipe-and-Filter, Repository.

Selected Pattern	[e.g., Three-layer MVC / Client-Server / Microservices]
Justification	[Why this pattern? Which verified requirements drove this choice?]
Traceability	[SRS requirement IDs that this architectural choice satisfies, e.g., SRS-REQ-007, SRS-REQ-012]

2.2 System Architecture Diagram

i Insert a high-level architecture diagram showing all major components/layers, the relationships and communication paths between them, external systems the software interacts with, and users. Label each component and each interface. Use a standard notation (UML component diagram, block diagram, or C4 model context/container diagrams). Attach the diagram file in the appendix as well.

[INSERT ARCHITECTURE DIAGRAM HERE]

Use Insert > Picture or paste from your diagramming tool (draw.io, Lucidchart, PlantUML, etc.)

Minimum height: 10 cm. Include a legend if notation is used.

Figure 2.1 Caption

[Architecture diagram title — e.g., 'Figure 2.1: Three-Layer Architecture of the Online Library System']

2.3 Component Description

i For each major component shown in the architecture diagram, provide a brief description. Use one row per component.

Component Name	Responsibility	Technology / Implementation
[Component 1]	[What does this component do? What is it responsible for?]	[Language, framework, or technology used]
[Component 2]	[Responsibility]	[Technology]
[Component 3]	[Responsibility]	[Technology]
[Add rows]	[as needed]	[]

2.4 Technology Stack

 List all technologies, languages, frameworks, libraries, and tools that will be used in the implementation, and justify each choice with reference to the requirements or design constraints.

Layer	Technology	Version	Justification
Frontend / UI	[e.g., React, HTML/CSS, Flutter]	[version]	[Why chosen? Requirement reference]
Backend / Logic	[e.g., Python/Django, Node.js, Java]	[version]	[Why chosen?]
Database	[e.g., MySQL, PostgreSQL, MongoDB]	[version]	[Why chosen?]
Authentication	[e.g., JWT, OAuth 2.0, session-based]	[version]	[Why chosen?]
Hosting / Deploy	[e.g., AWS, Heroku, local server]	[N/A]	[Why chosen?]
Version Control	[e.g., Git / GitHub]	[version]	[Team collaboration]
[Add layer]	[Technology]	[version]	[Justification]

3. Detailed Design

i Section 3 provides the detailed design of each major component. For each component, describe its internal structure, its interface (what it receives and what it returns), and its behavior. The level of detail should be sufficient for a developer to implement the component without ambiguity.

3.1 Module / Component Design

i Repeat section 3.1 for each major module or component. Copy the subsection block below and fill in the details for each one. Label them 3.1.1, 3.1.2, 3.1.3, etc.

3.1.1 [Component / Module Name]

Component name	[Full name of this module or component]
-----------------------	---

SRS traceability	[Which verified requirements does this component implement? List requirement IDs]
-------------------------	---

Responsibility	[In one sentence: what is the single responsibility of this component?]
-----------------------	---

Input / Preconditions

[What inputs does this component receive? From where? What state must the system be in before this component can operate?]

Output / Postconditions

[What does this component produce or return? What is the state of the system after this component completes?]

Processing Logic

[Describe the algorithm or logic this component uses. Use pseudocode, numbered steps, or a flowchart. Be specific enough that a developer can implement it without guessing.]

Error Handling

[What errors or exceptions can occur? How does this component detect them? What does it do when they occur — log, retry, propagate, display message?]

3.1.2 [Component / Module Name]

 Copy the fields above and fill in for the next component. Add as many 3.1.x subsections as needed — one per major component.

[Repeat 3.1.x block for each additional component]

3.2 User Interface Design

 Describe or show the key screens or interface elements of the system. For each screen, provide a wireframe or mockup (insert as an image) and describe what the screen does, what inputs it accepts, and what outputs or transitions it triggers. If the system has no UI (e.g., an API or background service), write N/A and describe the interface in Section 3.3 instead.

3.2.1 [Screen / Page Name]

Screen name *[e.g., Login Screen / Dashboard / Registration Form]*

Triggered by *[What user action or system event displays this screen?]*

SRS traceability *[Which requirement(s) does this UI element satisfy?]*

[INSERT WIREFRAME / MOCKUP HERE]

Paste from Figma, Balsamiq, Canva, or hand-drawn scan. Label all UI elements.

UI Element Description

[Describe each element on the screen: input fields (name, type, validation), buttons (label, action triggered), labels, error messages, and navigation options.]

Add More Screens

Add one 3.2.x subsection for each significant screen or page in your system. Prioritize screens that interact with critical business logic or implement verified security/validation requirements.

3.3 Interface Design

i Describe the interfaces between internal components and between the system and external systems, users, or devices. This includes API endpoints, database interfaces, and external service integrations.

3.3.1 Internal Component Interfaces

i For each pair of components that communicate, describe the interface: what data is sent, in what format, and what is expected in return.

Caller Component	Called Component	Data Passed	Expected Return
[Component A]	[Component B]	[Parameters / payload]	[Return value / response]
[Add row]	[]	[]	[]

3.3.2 External System Interfaces

i Describe any external systems, APIs, or services your system integrates with.

External System	Purpose	Interface Type	SRS Reference
[e.g., Google Maps API]	[Map display]	[REST API / HTTPS]	[REQ-014]
[e.g., Email service]	[Send notifications]	[SMTP]	[REQ-022]
[Add row]	[]	[]	[]

4. Data Design

i Section 4 describes how data is structured and stored. This includes the database schema or data model, entity relationships, and any data validation rules enforced at the data level.

4.1 Entity-Relationship Diagram (ERD)

i Insert the ERD showing all entities, their attributes, and the relationships between them. Use standard ERD notation (Crow's Foot or UML class diagram style). Label all entities, primary keys (PK), foreign keys (FK), and cardinalities (1:1, 1:N, M:N).

[INSERT ENTITY-RELATIONSHIP DIAGRAM HERE]
Label all entities, attributes, primary keys (PK), foreign keys (FK), and relationship cardinalities.

4.2 Data Dictionary

i Define every entity (table) and its attributes. For each attribute, specify: name, data type, constraints, and description. Add one table per entity/table in your data model.

Entity / Table: [Entity Name]

Attribute Name	Data Type	Constraints	Key	Description
[attribute_id]	[INT]	NOT NULL, AUTO_INCREMENT	PK	[Unique identifier]
[attribute_name]	[VARCHAR(100)]	NOT NULL		[Description of this attribute]
[foreign_key]	[INT]	NOT NULL	FK	[References [OtherEntity](id)]
[created_at]	[DATETIME]	NOT NULL		[Timestamp of record creation]
[Add rows]	[]	[]	[]	[]

Add More Entities
Copy the table above and add one data dictionary table per entity/table in your data model. Ensure every table in your ERD has a corresponding data dictionary entry.

5. Security Design

i Describe the security mechanisms built into the design. Every security-related requirement in the verified SRS must be addressed here. If your system has no authentication, data protection, or access control requirements, justify that decision explicitly.

5.1 Authentication and Authorization

Authentication Mechanism	[e.g., Username + password with bcrypt hashing / JWT tokens / OAuth 2.0 / session-based]
---------------------------------	--

SRS Traceability	[Requirement ID(s) from the verified SRS]
-------------------------	---

Authorization Model	[e.g., Role-based access control (RBAC) / Attribute-based / no multi-role]
----------------------------	--

Roles Defined	[List all user roles and their permissions — e.g., Admin: full access; User: read own data only]
----------------------	--

Security Design Detail

[Describe how authentication and authorization are implemented. Where are tokens stored? How are passwords hashed? How are roles enforced — at the controller level, middleware, or database?]

5.2 Input Validation and Injection Prevention

Validation Location	[Server-side, client-side, or both?]
----------------------------	--

Validation Approach	[e.g., Allowlist validation, parameterized queries for all DB access, ORM]
----------------------------	--

Threats Addressed	[e.g., SQL injection via parameterized queries; XSS via output encoding; CSRF via CSRF tokens]
--------------------------	--

Validation Rules

[List specific validation rules for key inputs — e.g., Email: RFC 5322 format, max 255 chars; Password: min 8 chars, at least one uppercase, digit, and special character; Age: integer 0–120]

5.3 Data Protection

Data Encrypted in Transit	[Protocol — e.g., HTTPS/TLS 1.3 enforced for all communications]
Data Encrypted at Rest	[e.g., Sensitive fields encrypted using AES-256; or 'not applicable for this project scope']
Sensitive Data Identified	[List data elements that are sensitive: passwords, payment info, personal data, health data]
Logging Policy	[What is logged? What is explicitly NOT logged (passwords, tokens, PII)?]

6. Requirements Traceability Matrix

i The Requirements Traceability Matrix (RTM) links every verified requirement from the SRS to the design element(s) that implement it. Every requirement must appear in this matrix. If a requirement is not addressed by any design element, it is a design gap that must be resolved before implementation begins.

Completeness Check

Count the total number of verified requirements in your SRS. Count the rows in this RTM. They must be equal. A missing requirement means part of the verified specification has no design — this is a critical defect in the design document.

Req. ID	Requirement Summary	Design Element(s)	Section Ref.	Status
REQ-001	[Requirement summary from SRS]	[Component / module name]	§ 3.1.1	Addressed
REQ-002	[Requirement summary]	[Design element]	§ 3.2.1	Addressed
REQ-003	[Requirement summary]	[Design element]	§ 4.2	Addressed
REQ-004	[Security requirement]	[Auth mechanism]	§ 5.1	Addressed
REQ-005	[Requirement summary]	[Design element]	§ 3.3.1	Addressed
[...]	[Add one row per verified requirement]	[]	[]	[]

RTM Summary

Total verified requirements in SRS	[Enter number]
Requirements addressed in this SDS	[Enter number]
Requirements NOT yet addressed (gaps)	[Enter number — must be 0 for complete design]
Coverage percentage	[(Addressed / Total) × 100%]

7. Design Assumptions and Constraints

i Document all assumptions made during design (things believed to be true but not verified) and all constraints (limitations imposed by the technology, environment, time, or requirements).

7.1 Assumptions

i List every assumption that, if wrong, would invalidate part of the design. For each assumption, state what would need to be redesigned if the assumption proves incorrect.

#	Assumption	Impact if Wrong
1	[State the assumption clearly]	[What part of the design would need to change?]
2	[State the assumption clearly]	[What part of the design would need to change?]
3	[State the assumption clearly]	[What part of the design would need to change?]

7.2 Constraints

i List all constraints on the design — technical constraints (must run on a specific platform), resource constraints (limited development time or budget), or regulatory constraints (must comply with a specific standard or law).

Type	Constraint	Design Response
Technical	[Describe the constraint]	[How does the design accommodate this constraint?]
Resource	[Describe the constraint]	[How does the design accommodate this constraint?]
Regulatory / Policy	[Describe the constraint]	[How does the design accommodate this constraint?]
Environmental	[Describe the constraint]	[How does the design accommodate this constraint?]

8. Appendices

 Include any supplementary material that supports but does not fit within the main sections. Label each appendix alphabetically (Appendix A, Appendix B, etc.).

Appendix A — Architecture Diagram Source File

Location
[Provide the filename and storage location (e.g., GitHub repository path, shared drive link) for the original architecture diagram file so reviewers can access the editable version.]

Appendix B — UML Diagrams

 Attach any UML diagrams produced during design that are not embedded in the main body — sequence diagrams, class diagrams, activity diagrams, state machine diagrams. Label each with a figure number and caption.

UML Diagrams
[List each diagram: Figure B.1 — Class Diagram; Figure B.2 — Sequence Diagram: User Login Flow; Figure B.3 — State Diagram: Order Status; etc. Insert diagrams below each entry.]

Appendix C — Design Review Checklist

 Complete this checklist before submitting the SDS. All items should be checked before submission.

#	Checklist Item	Status
1	Every verified requirement from the SRS appears in the RTM	[☐] / [☒]
2	RTM coverage percentage is 100%	[☐] / [☒]
3	Architecture diagram is present, labeled, and has a caption	[☐] / [☒]
4	Every component in the architecture diagram has a description in Section 2.3	[☐] / [☒]
5	Every screen/page has a wireframe and description in Section 3.2	[☐] / [☒]
6	ERD is present, labeled, and all entities have data dictionary entries in Section 4.2	[☐] / [☒]
7	All security requirements from SRS are addressed in Section 5	[☐] / [☒]

8	All placeholder text [in brackets] has been replaced with actual content	[☐] / [☒]
9	Revision history is up to date with the current version number and date	[☐] / [☒]
10	Document has been spell-checked and reviewed by all team members	[☐] / [☒]

Appendix D — Glossary of Design Patterns

For reference, the following design patterns are commonly applied in student projects at this level:

Pattern	Description	When to Use
MVC (Model-View-Controller)	Separates data (Model), display (View), and logic (Controller) into three distinct layers	Web applications; when UI and logic must be independently modifiable
Repository Pattern	Abstracts data access behind an interface so that the rest of the application does not depend on a specific database technology	When data source may change; improves testability
Singleton	Ensures a class has only one instance and provides a global access point to it	Configuration objects, database connection pools
Observer	Defines a one-to-many dependency so that when one object changes state, all dependents are notified automatically	Event-driven systems, notification features
Factory	Defines an interface for creating objects but lets subclasses decide which class to instantiate	When object creation logic is complex or conditional
Facade	Provides a simplified interface to a complex subsystem	Hiding complex API integrations behind a clean internal interface

References

- IEEE. (2009). IEEE 1016-2009: IEEE Standard for Information Technology — Systems Design — Software Design Descriptions. IEEE.
- IEEE Computer Society. (2014). SWEBOK v3.0: Guide to the Software Engineering Body of Knowledge. IEEE Computer Society Press.
- Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill.
- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
- NIST. (2020). NIST SP 800-218: Secure Software Development Framework (SSDF). National Institute of Standards and Technology.